

TECHNICAL DESIGN STUDY

FORGE

Beyond **grind** through structural search and certified algebra

Concrete algorithms, exact prototypes, and a reproducible evaluation package

Prepared for Vladimir Reshetnikov

14 September 2026

What is delivered. A proposed Lean automation architecture and six executable Python search components, with exact certificate checking, generated Lean replay candidates, test code, and machine-readable results.

What was measured. All **275 test instances** passed. All **198 constructed component-benchmark instances** produced certificates that passed a separate, standard-library-only recheck.

What was not measured. No Lean executable was available in the authoring environment. The Lean files have **not been compiled**. The complete **forge** tactic is a design, not an implemented product, and no head-to-head advantage over **grind** is claimed from these measurements.

Baseline source snapshot: Lean v4.34.0. Candidate replay environment: the exact Mathlib revision recorded in the package. Search is replaceable; acceptance must remain proof-producing.

Abstract

A substantially more capable successor to `grind` should not merely increase its search limits or rename a portfolio of existing tactics. It should retain `grind` as a fast, trusted proof-producing core while addressing three different sources of failure: missing proof structure, missing intermediate mathematical facts, and missing existential constructions. This report proposes FORGE, a typed, scope-aware coordination layer around structural AND/OR search, goal-directed theorem application, induction with generalization, witness synthesis, and exact nonlinear certificates. The concrete arithmetic design includes rational quadratic decomposition, finite-cone linear programming with exact repair, multivariate Bernstein subdivision, and univariate square-factor/Sturm certificates that tolerate exact zeros. Polynomial recurrence synthesis and affine witness synthesis provide executable examples of structure-producing automation.

The report specifies algorithms, soundness arguments, integration contracts, resource accounting, libraries, trust modes, and an implementation sequence. Six components are prototyped and tested in Python; stored certificates can be rechecked without importing SciPy, NumPy, or SymPy. The experiments isolate useful mechanisms but are deliberately not presented as evidence that the unimplemented Lean tactic outperforms `grind`. A separate evaluation protocol explains how to establish that claim fairly.

Contents

1	The proposal in one page	4
1.1	Two meanings of “more powerful”	4
2	A strong baseline, not a straw man	5
2.1	Where new coverage can realistically come from	5
2.2	Relationship to existing automation	6
3	Architecture: structural search around a shared fact store	6
3.1	The layers	6
3.2	A concrete worker contract	6
3.3	Typed views and scope keys	7
3.4	State restoration and proof transport	7
3.5	Event-driven cooperation	8
3.6	An explicit scheduler	8
4	Directed theorem use and constructive witnesses	9
4.1	Backward application complements forward matching	9
4.2	Witness grammars and universal validation	9
4.3	Executable affine synthesis	9
4.4	Worked affine example	10
5	Induction, generalization, and invariant synthesis	10
5.1	Selecting a scheme from recursive structure	10
5.2	A motivating accumulator	11

5.3	A bounded proof-progress measure	11
5.4	Exact polynomial recurrence synthesis	11
5.5	Scaling beyond the prototype	12
6	A certificate calculus for nonlinear inequalities	12
6.1	The acceptance language	12
6.2	What Lean replay would do	13
7	Algorithm A: exact rational quadratic decomposition	13
7.1	Matrix construction	13
7.2	Pseudocode and cost	14
7.3	Why this adds a useful mechanism	14
8	Algorithm B: finite-cone search with exact repair	15
8.1	Candidate construction	15
8.2	Repair instead of rounding a proof	15
8.3	A domain-dependent example	15
8.4	A concrete path to general SOS search	16
9	Algorithm C: exact Bernstein subdivision on boxes	16
9.1	Coefficient formula	16
9.2	The subdivision certificate	17
9.3	Limits and the exact-zero obstruction	17
9.4	A nontrivial box example	18
10	Algorithm D: univariate certificates that preserve zeros	18
10.1	Square-factor reduction	18
10.2	The checked Sturm chain	18
10.3	Soundness and fragment completeness	19
10.4	Why this worker is useful alongside Bernstein search	19
11	Theory exchange, guarded normalization, and equality search	19
11.1	Certified product envelopes	19
11.2	A complete cross-theory proof shape	20
11.3	Bounded equality search	20
11.4	What a richer system still needs	20
12	Libraries, modules, and implementation boundaries	20
12.1	Use existing libraries for what they already do well	20
12.2	A suggested Lean module layout	21
12.3	A proposed user interface	21

12.4 The external certificate protocol	22
13 Trust: what exactly is being checked?	22
13.1 Three distinct layers	22
13.2 Axiom auditing is part of acceptance	23
13.3 Diagnostic holes are not accepted proofs	23
13.4 The proof-cost budget	23
14 Executed experiments	23
14.1 Environment and reproducibility	23
14.2 Unit, mutation, and independent tests	24
14.3 Constructed component benchmark	24
14.4 What the results establish	25
15 How to evaluate an actual Lean implementation fairly	25
15.1 Baselines and information access	25
15.2 A useful corpus design	25
15.3 Resources and metrics	26
15.4 Ablations that would test the actual design	26
16 Implementation sequence and acceptance gates	26
16.1 Milestone 1: trustworthy orchestration	26
16.2 Milestone 2: exact algebraic replay	26
16.3 Milestone 3: structural planning and witnesses	27
16.4 Milestone 4: domain-aware positivity	27
16.5 Milestone 5: efficient cooperation	27
16.6 Milestone 6: optional broader search	27
17 Reproducing and extending this package	27
17.1 Python commands	27
17.2 Lean candidate compilation	28
17.3 Archive map	28
18 Conclusion	29
References	30

1 The proposal in one page

FORGE is a *proof-producing automation system with a structural search layer*, not a new foundational logic. It preserves the existing Lean kernel and treats search engines, learned rankings, computer-algebra systems, and numerical solvers as fallible advisers. Its intended user-facing entry point is a terminal tactic, with an explanatory variant that emits a smaller replay script.

The main design choice is to separate two tasks that are often conflated. A *structural planner* chooses a proof shape: introduce a binder, construct a witness, apply a theorem, prove an auxiliary lemma, or use an induction principle. A *local saturation engine* derives consequences in the resulting context. **grind** is an excellent candidate for the second role. The proposed improvement is to supply proof structure and certified facts that are not available from local saturation alone, and to feed useful consequences back rather than restarting every tool from an unchanged goal.

For example, a nonlinear worker may prove $0 \leq p(x, y)$, combine it with an existing $p(x, y) \leq 0$, and obtain $p(x, y) = 0$. The equality is then useful to congruence closure even when the final target is an equality between uninterpreted function applications. An induction planner may generalize an accumulator, synthesize its polynomial invariant, and invoke **grind** only on the base and step obligations. A witness worker may construct a parametric solution of a linear system and leave integrality or bound obligations to other workers. In all three cases, the benefit comes from *coordination around checked intermediate claims*.

The highest-priority implementation is therefore a small system with strong interfaces: a baseline-preserving entry point, typed reification and certificate replay, a scope-safe fact store, polynomial certificates, a bounded induction planner, and a witness synthesizer. General higher-order search, semidefinite programming, and learned scheduling are optional later extensions. They are not prerequisites for demonstrating useful new coverage.

1.1 Two meanings of “more powerful”

The desired outcome is substantially higher automatic coverage on useful Lean developments. That does not mean uniform speed improvement, logical completeness, or success-set inclusion at a fixed total budget. Those are different claims.

Definition 1.1. For a tactic implementation T , an environment E , a goal G , and a resource allocation B , write $T(E, G; B) \downarrow$ when it returns a proof accepted under the selected trust policy within that allocation.

FORGE should offer two modes. In *compatibility mode*, it first runs the stock **grind** invocation, with the same input, options, and solver budget, before running new machinery. In *fixed-budget mode*, it partitions a single budget among workers and may sacrifice some baseline successes in exchange for others.

Proposition 1.2 (Conditional baseline preservation). *Assume the stock run is reproduced deterministically with identical inputs and resources, and its successful proof is returned immediately. If compatibility mode allocates that full run before spending additional extension resources, then every success of the stock invocation is also a success of compatibility mode.*

Proof. The first stage is the stock invocation itself. A success exits before any extension can change its state or consume its reserved allocation. $\square$

This is a scheduling property, not a superiority result at equal cost. Process startup, memory pressure, imports, and global wall-clock limits can invalidate an operational interpretation unless

controlled. A fair experiment must therefore report both modes. Failure-triggered interventions also have recent precedent in research on `grind`; the cascade itself is not claimed as a new invention here [14].

2 A strong baseline, not a straw man

The reference describes `grind` as cooperating proof-producing engines organized around contradiction, congruence closure, constraint propagation, E-matching, guided case analysis, and theory solvers. This is already a shared-fact architecture, not a sequence of independent tactic calls [1]. A credible successor must reuse those strengths.

The v4.34.0 configuration source is particularly important because it prevents several obsolete claims about missing functionality. Table 1 records capabilities that must be counted as baseline features [2, 3].

Area	Already present in the inspected source	Consequence for this design
Equality	Congruence reasoning, function-valued congruence, extensionality controls, injectivity support	Do not advertise ordinary e-graphs or function equalities as new.
Arithmetic	Ring and field-related algebra, linear arithmetic, integer arithmetic, associative/commutative reasoning, order support	Add certified inequality reasoning and structure, not a duplicate ring normalizer.
Quantifiers and search	E-matching, term-generation limits, case splits, lookahead, model-based theory combination	Distinguish goal-directed construction from existing instantiation and splitting.
Libraries	Configurable library suggestions and local-definition use	Improve retrieval and obligation-driven use; do not claim retrieval is wholly absent.
Engineering	Saved states, proof-related diagnostics, explicit search limits	Reuse existing machinery behind a versioned adapter.

Table 1: Baseline features from the pinned Lean configuration and implementation sources. The table is not a claim of completeness for any theory.

For reproducible comparisons, the source defaults include nine search-branch splits, five E-matching rounds before a split, generation bounds of eight, and a thousand E-matching instances per branch. The source exposes many additional limits. A future benchmark should record effective options, not merely write “default `grind`”, and should include a reasonable tuned baseline [2].

2.1 Where new coverage can realistically come from

The target is not “everything `grind` cannot prove”. It is a collection of mechanisms with a plausible cost/coverage tradeoff:

1. **Proof structure:** select induction principles, generalize dependent state, construct existential witnesses, and establish auxiliary lemmas before using them.
2. **Missing algebraic consequences:** discover hidden sums of squares, domain-dependent polynomial positivity, and exact zero information that can be shared with other engines.
3. **Directed obligation solving:** choose theorem applications from the conclusion backward, create their side conditions explicitly, and use successful side conditions to activate guarded

forward reasoning.

These are proposed extensions, not assertions that every example requiring one will fail under all existing `grind` configurations or library annotations. Some will already be solved by `grind`, `nlinarith`, Aesop, or a suitable theorem. Only actual controlled runs can identify new coverage on a particular corpus.

2.2 Relationship to existing automation

Aesop already provides rule-based AND/OR proof search, distinguishes normalization and safe/unsafe rules, and uses best-first exploration. Its documented induction examples still perform induction explicitly. It is a useful source of search infrastructure, not a feature to rediscover [4]. Duper supplies proof-producing saturation over supplied facts; its README describes manual premise selection and a configurable portfolio. It is a complementary clause-search worker, especially after a relevance filter has selected a small premise set [5].

Mathlib’s `linarith` separates certificate search from proof reconstruction, and `nlinarith` adds specific nonlinear preprocessing. That is an appropriate architectural precedent, but it is not a full general sums-of-squares solver [6]. Foundational checking of numerically discovered SOS certificates also has a substantial history; Harrison’s HOL Light work explicitly separates sophisticated search from relatively straightforward verification [9].

The old `lean-egg` project should not be a mandatory new dependency: its repository says it is no longer developed and directs users toward `grind` [13]. The useful idea is bounded, proof-producing equality search, not allegiance to that particular package.

3 Architecture: structural search around a shared fact store

3.1 The layers

At the outer layer, an AND/OR search graph represents structural choices. An AND node contains obligations that must all be proved, such as the premises of a theorem application or all cases of an induction. An OR node contains alternative proof plans. Each open leaf has a local context and a target.

At a leaf, a coordination layer exposes certified facts and typed views to specialized workers. `grind` maintains its own efficient internal equality and theory state. The initial integration should use supported terminal tactic calls and explicit proved lemmas. A later, version-pinned adapter may subscribe directly to internal events and add proved facts without restarting saturation. The latter is an engineering optimization and must not be assumed to exist in the prototype.

The acceptance layer reconstructs a Lean proof of each returned proposition, checks scope and dependencies, and ultimately constructs a proof of the original goal. Numerical success, satisfiability status, an unverified certificate, and a plausible conjecture are never accepted as facts.

3.2 A concrete worker contract

The following is an interface specification, not compiled Lean source:

```

Worker.supports : GoalView -> SupportReport
Worker.step      : Snapshot -> DeltaFacts -> Budget -> WorkerResult

WorkerResult :=
| solved      (proof : CheckedProofRef)
| progress    (facts : Array ScopedFact)
               (obligations : Array ConditionalObligation)
               (resumeToken : OpaqueToken)
| candidate   (kind : CertificateKind) (payload : BoundedData)
| unknown     (reason : FailureReason)

ScopedFact := {
  proposition : Expr,
  proof       : Expr,
  scope       : ScopeId,
  dependencies: Array FactId
}

```

`CheckedProofRef` is a capability issued by the acceptance layer, not by an external process. An external solver can return only data or a candidate. A native Lean tactic worker may return a proof expression, but the coordinator still verifies its type, local-variable support, auxiliary declarations, and allowed axioms before committing it.

A conditional obligation is a directed dependency, not an assertion. For instance, cancellation of a in a field creates an obligation $a \neq 0$. Until that proof exists, the cancelled equality cannot enter the fact store. The result protocol must keep a conjectured fact visibly different from a proved fact even when both share the same proposition.

3.3 Typed views and scope keys

The system needs a small number of explicit views: equality/Boolean facts, affine constraints, polynomial constraints, recursive-definition equations, and existential goals. Each view stores both the reified object and a proof connecting it to the original Lean expression. An opaque subterm may be represented by a fresh polynomial variable, but it must retain the correct type and identity.

A suitable key contains the expression, its type, universe levels where relevant, the identity of the ordered ring or field instance, a local-context fingerprint, and a scope identifier. Printed text is not a safe key. Neither are expression pointers suitable for persistent caches across environments. Alpha-equivalence and definitional equality can improve sharing, but any metavariable assignments they create must remain local to the search branch.

For example, $x - y$ over $\mathbb{N}$ is truncated subtraction, whereas over $\mathbb{Z}$ or $\mathbb{R}$ it is ring subtraction. A syntactic translator that treats these identically is unsound. A `Fin n` expression is not automatically an integer with unrestricted arithmetic. A proposition about real division cannot be transported through a denominator without a suitable nonzero or sign guard. Typeclass instances must be part of the interpretation, not inferred from a convenient printed operator.

3.4 State restoration and proof transport

A failed worker must not leave metavariable assignments, postponed constraints, fresh hypotheses, or changed transparency settings in another branch. The inspected `grind` implementation already has saved-state machinery [3]; FORGE should use a thin adapter rather than copy its internal data structures blindly.

There is a subtler issue: a successful worker may introduce auxiliary theorem declarations

and return a proof referring to them. Restoring the environment and retaining only the final expression would create dangling references. A commit must therefore preserve the required declaration closure, after checking it, or reconstruct a self-contained proof term in the parent environment. External results must be re-elaborated against the exact destination context rather than transporting free-variable identifiers between unrelated goals.

Facts proved using a branch hypothesis can be shared only within that branch or a descendant. To share them upward, discharge the hypothesis and transport an implication, or combine proofs from a complete case split. This rule applies just as strongly to numerical bounds, inferred equalities, and cached certificates as to ordinary theorem applications.

3.5 Event-driven cooperation

Workers should subscribe to deltas rather than repeatedly scan the entire context. Typical events are: a new bound on an arithmetic expression, a new equality joining two term classes, a proved side condition, an instantiated theorem, and a changed target after structural decomposition.

The coordinator normalizes a returned fact, checks whether it adds information, stores its proof, and notifies only relevant subscribers. For example, changing an interval bound invalidates or strengthens a Bernstein search problem; learning a polynomial equality may reduce the dimension of a subsequent positivity certificate; proving an existential witness’s denominator nonzero activates its algebraic replay.

Logical circularity and scheduling cycles must be distinguished. A worker waiting for a fact that another worker might establish is an ordinary dependency cycle; it may require a different search action. It does not permit both workers to assume their outputs. Track these dependencies as strongly connected components and either pursue an independent obligation, perform a justified case split, or return a diagnostic explaining the blockage.

3.6 An explicit scheduler

The initial implementation should use deterministic cost-aware scheduling, not require a trained model. Each task has a relevance estimate, expected information gain, recent cost, age, and a resource ceiling. A simple score is

$$s(a) = w_r R(a) + w_g G(a) + w_a A(a) - w_c \hat{C}(a),$$

with the weights fixed in the experiment manifest. This is only a heuristic. Reserve a minimum quota for every eligible worker class so that a high-scoring stream of cheap actions does not starve structural search.

```

solve(goal, mode, budgets):
  if mode == compatibility:
    run_exact_stock_grind(goal, budgets.stock)
    if accepted: return proof
    restore_original_goal()
  agenda := initial_structural_and_local_tasks(goal)
  while resources_remain:
    action := choose_with_reserved_worker_quotas(agenda)
    snapshot := save_full_branch_state()
    result := run_bounded(action)
    restore_or_commit_only_checked_changes(snapshot, result)
    if original_goal_has_checked_proof(): return proof
    enqueue_new_obligations_and_relevant_delta_subscribers()
  return Unknown(with_structured_diagnostics)

```

Heartbeats alone are insufficient for external numerical code. Maintain separate limits on wall time, memory, instantiated theorems, polynomial monomials, rational bit lengths, certificate size, induction depth, and replay work. External workers should run in bounded child processes without network access. A process timeout returns `Unknown` and cannot create a theorem.

Finite budgets imply incompleteness. Fair worker quotas do not make unrestricted Lean proof search complete. They make the search policy interpretable and reduce avoidable starvation.

4 Directed theorem use and constructive witnesses

4.1 Backward application complements forward matching

Use a conclusion index over theorem types, initially a discrimination tree keyed by the target's head symbol and normalized type structure. Retrieve a bounded list of candidates, instantiate universes, and unify the theorem conclusion with the target. Successful unification creates premises as AND obligations and instantiation choices as OR alternatives. Explicit dependencies prevent the theorem's conclusion from becoming available before its premises are proved.

Suppose a theorem has the shape

$$\forall x, P(x) \rightarrow Q(f(x)).$$

For a goal $Q(f(u))$, backward application immediately proposes the obligation $P(u)$ and introduces useful terms even when no suitable trigger was present in the initial forward state. For a goal $Q(t)$ with arbitrary t , it does *not* justify inventing an inverse of f . The system must unify t with an application already known to be equal, synthesize a candidate u and prove $f(u) = t$, or decline the application.

Premise retrieval should combine the target with currently blocked obligations, not just a bag of symbols from the original goal. A theorem resolving a denominator guard can be more useful than one mentioning the final function symbol. Existing library-suggestion support remains part of the baseline; the proposed addition is its integration with explicit obligation production and feedback.

4.2 Witness grammars and universal validation

For a goal $\forall x \exists y, \Phi(x, y)$, the witness worker seeks a term $t(x)$ and creates the universal obligation $\Phi(x, t(x))$. The search grammar should be type-directed and size-bounded: local variables, constructors, projections, selected recursive calls, affine combinations, and guarded division or remainder operations. Piecewise terms are permitted only with exhaustive, proved branch conditions.

Counterexample-guided search can quickly reject candidates using samples or a model of an abstraction. Such rejection improves search, but passing samples is never evidence sufficient to prove the universal obligation. A candidate can be used only after Lean proves the resulting statement. Over `Fin n`, witness construction includes a bound proof; over `Nat`, it includes nonnegativity where a signed intermediate representation is used.

4.3 Executable affine synthesis

Status: Implemented and tested in Python; candidate Lean replay emitted.

The implemented fragment solves

$$Aw = Bx + c$$

by searching for an affine witness $w = Wx + d$. Coefficient matching gives the exact linear systems

$$AW = B, \quad Ad = c.$$

The oracle performs rational reduced row-echelon elimination, independently for each column of B and for c . Free variables are set to zero. The checker multiplies the proposed matrices again and compares every entry exactly. No sampled values of x are involved.

Over $\mathbb{Q}$, this is complete for the affine-witness template, ignoring resource bounds. The implemented integer mode merely requires every coefficient of W and d to be integral. It is intentionally incomplete: a different choice of free variables might yield an integral solution even when the default rational solution does not.

A production integer worker should compute a Smith normal form $UAV = D$ with unimodular U, V , validate the transformation identities and unimodularity certificates, and solve the divisibility conditions in $Dz = U(Bx + c)$. This may require assumptions on x , rather than an unconditional affine term. It is a specified extension, not part of the executable prototype.

For example, $2w = x$ has the rational witness $x/2$, but no integer witness for every integer x . An integer solver must obtain a parity assumption or fail. It must never reinterpret rational division as integer arithmetic silently.

4.4 Worked affine example

The system

$$u + 2v = 3x - y + 5, \quad v = 2x + 4y - 3$$

has the integral affine solution

$$v = 2x + 4y - 3, \quad u = -x - 9y + 11.$$

A replay simply constructs these expressions and proves the two identities with `ring`. The corresponding candidate script is included in `lean/GeneratedReplay.lean`. This is a construction, not just a contradiction proof of an existential statement.

5 Induction, generalization, and invariant synthesis

5.1 Selecting a scheme from recursive structure

The induction planner starts from recursive calls visible in the target and relevant hypotheses. For each, it records the recursive arguments, arguments that change across calls, and the equations that the definition generates. It then proposes a bounded collection of structural or functional induction schemes. User-specified schemes take priority. Blind induction on every natural variable is not acceptable.

Each candidate plan has an explicit Lean motive, an induction principle, generalized variables, and recursive hypotheses available in each case. Indexed inductive families require motives that retain their indices and dependent hypotheses. A case split on an inductive value does not automatically supply an induction hypothesis; confusing the two is a design error.

Generalization follows a dependency closure. If an accumulator changes in a recursive call, generalize it; also revert hypotheses and local definitions whose types or values depend on it, then reintroduce them where appropriate. Candidate generalization sets are ordered by size and capped. This does not solve every induction problem, but it targets a common and mechanically detectable failure mode.

5.2 A motivating accumulator

Consider the recursive function

$$A(0, a) = a, \quad A(n+1, a) = A(n, a + 2n + 1).$$

The desired theorem is $A(n, a) = a + n^2$. Inducting on n while keeping a fixed gives a hypothesis about $A(n, a)$, whereas the recursive call needs $A(n, a + 2n + 1)$. Generalizing a first gives

$$\forall a, A(n, a) = a + n^2,$$

which specializes to the changed accumulator. The step reduces to

$$a + 2n + 1 + n^2 = a + (n + 1)^2.$$

The packaged `StructuralReplay.lean` gives an explicit candidate Lean proof over integer accumulators. The important automation problem is choosing the generalization, not performing the final ring calculation.

5.3 A bounded proof-progress measure

After introducing a case, identify the recursive call occurrences that could match the induction hypotheses. Prefer rewrites that decrease constructor mismatch, unmatched accumulator updates, and target size, in that order. A lexicographic measure provides a practical way to avoid repeatedly unfolding a recursive function in the wrong direction.

When this fails, an auxiliary lemma may be proposed by anti-unifying a stuck term with an induction-hypothesis pattern or by introducing a polynomial invariant template. The proposed lemma becomes a separate proof obligation. It is never assumed in its own proof. General mutual or cyclic induction would require a distinct justified proof rule and is outside the initial implementation.

This planner is specified at algorithm level but is not implemented in the Python prototype. What is implemented is the invariant-discovery subproblem below.

5.4 Exact polynomial recurrence synthesis

Status: Implemented and tested in Python.

Let a sequence be defined by

$$S(0, \mathbf{a}) = I(\mathbf{a}), \quad S(n+1, \mathbf{a}) = S(n, \mathbf{a}) + P(n, \mathbf{a}),$$

where I, P have rational coefficients and I is independent of n . Search for

$$F(n, \mathbf{a}) = \sum_{|\alpha| \leq D} c_\alpha(n, \mathbf{a})^\alpha.$$

Substitute this ansatz into the base and step identities

$$F(0, \mathbf{a}) = I(\mathbf{a}), \tag{1}$$

$$F(n+1, \mathbf{a}) - F(n, \mathbf{a}) = P(n, \mathbf{a}). \tag{2}$$

Equating coefficients gives a rational linear system in the unknown c_α . The search raises D until it finds a solution or reaches a cap. The checker then recomputes both identities independently from the proposed polynomial.

Proposition 5.1 (Recurrence certificate). *If a polynomial F satisfies (1) and (2), then $S(n, \mathbf{a}) = F(n, \mathbf{a})$ for every $n \in \mathbb{N}$ and every admissible parameter tuple $\mathbf{a}$.*

Proof. The base identity proves the assertion at zero. Assuming it at n , substitute it into the recurrence and use (2) to obtain the assertion at $n + 1$. $\square$

The algorithm is complete within each finite polynomial template because exact linear-system solving decides whether its coefficient equations are consistent. For rational polynomial increments, a polynomial antidifference exists; nevertheless, the implemented search may return **Unknown** when its degree cap is too small. General recurrences involving products with S , branching updates, or nonlinear state transformations are not accepted by this interface.

For $P(n) = n^2$ and $I = 0$, the synthesized result is

$$F(n) = \frac{n^3}{3} - \frac{n^2}{2} + \frac{n}{6} = \frac{n(n-1)(2n-1)}{6}.$$

The experiment includes power sums from exponent zero through eight and 24 parameterized polynomial recurrences. The generated Lean candidates instantiate natural-number induction and replay the identities over $\mathbb{Q}$; these scripts have not been compiled in this environment.

5.5 Scaling beyond the prototype

For m variables and total degree D , there are $\binom{m+D}{D}$ template monomials. The small implementation enumerates a Cartesian exponent grid and filters it; this is simple but wastes work. A production version should enumerate bounded compositions directly, exploit sparse coefficient matrices, and use fraction-free or modular elimination to limit rational coefficient growth. None of these optimizations changes the acceptance condition: the base and step identities must still be checked exactly.

6 A certificate calculus for nonlinear inequalities

6.1 The acceptance language

The nonlinear worker initially targets polynomial statements over $\mathbb{R}$ with exact rational coefficients. Variables may stand for opaque real subexpressions; additional relationships between them must be explicitly supplied as proved constraints. The same representation should not be reused over arbitrary rings with an assumed real ordering.

Normalize a target inequality to $p(\mathbf{x}) \geq 0$, hypotheses to $g_i(\mathbf{x}) \geq 0$, and equalities to $h_j(\mathbf{x}) = 0$. The general certificate accepted by the prototype is

$$p = \sum_{k=1}^s c_k q_k^2 \prod_{i=1}^m g_i^{a_{ki}} + \sum_{j=1}^r u_j h_j, \quad c_k \in \mathbb{Q}_{\geq 0}, \quad a_{ki} \in \mathbb{N}. \quad (3)$$

The q_k, u_j are rational polynomials. The checker is given the original p, g_i, h_j independently of the certificate. It reconstructs the right-hand side exactly and requires polynomial identity.

Proposition 6.1 (Cone-certificate soundness). *If (3) holds, every point satisfying $g_i \geq 0$ and $h_j = 0$ satisfies $p \geq 0$.*

Proof. Each square is nonnegative, each hypothesis product is nonnegative, and each coefficient c_k is nonnegative. Every equality-multiplier term vanishes. Summing proves the claim. $\square$

This elementary proof explains why search can be much more complicated than checking. The certificate language is a bounded preordering-style cone with equality-ideal terms. Restricting products to individual g_i gives a smaller family; fixing the possible q_k gives a finite cone. These distinctions matter: none should be called an unrestricted positivity decision procedure.

Strict inequalities need explicit strictness information. One sufficient certificate is an identity of the same form for $p - \varepsilon$ with a rational $\varepsilon > 0$. It is not necessary in general: a polynomial can be strictly positive on an unbounded domain while having infimum zero. Another route is a term whose coefficient and factors are provably strictly positive at every admissible point. The proof layer must preserve the strict inequality rather than silently replacing it with nonnegativity.

6.2 What Lean replay would do

For small certificates, generate an explicit expression built from rational coefficients, squares, hypothesis products, and equality multipliers. Establish the sign of each term using theorem applications or `positivity`, then establish the polynomial identity using `ring`. Equality terms are rewritten to zero using the supplied proofs. Mathlib provides the relevant tactic infrastructure [7, 8].

For larger certificates, define a reflected polynomial language and prove a generic evaluation theorem once. A Boolean checker can then be reduced in Lean without trusting a floating-point result. The full reified checker and its soundness theorem are implementation work, not included as a completed Lean library in this package.

In both cases, the connection between the reified variables and the original expression is part of the proof. A correct polynomial identity about the wrong reification is not a proof of the user's goal. Likewise, a certificate must identify the actual hypotheses used, not just provide polynomial strings that resemble them.

7 Algorithm A: exact rational quadratic decomposition

Status: Implemented, tested, and used to generate Lean replay candidates.

7.1 Matrix construction

For a rational quadratic polynomial in n variables, let

$$z = (1, x_1, \dots, x_n)^T, \quad p(\mathbf{x}) = z^T Q z,$$

where Q is the unique symmetric matrix obtained by placing constant and square coefficients on the diagonal and half of each mixed or linear coefficient in the corresponding symmetric off-diagonal entries.

Theorem 7.1 (Quadratic fragment). *A rational quadratic polynomial is nonnegative on all of $\mathbb{R}^n$ if and only if its homogenized symmetric matrix Q is positive semidefinite. In that case it admits a certificate*

$$p = \sum_k d_k \ell_k^2$$

with nonnegative rational d_k and rational affine linear forms ℓ_k .

Proof. If Q is positive semidefinite, the first assertion is immediate. Conversely, for a vector $(t, \mathbf{y})$ with $t \neq 0$,

$$(t, \mathbf{y})^T Q (t, \mathbf{y}) = t^2 p(\mathbf{y}/t) \geq 0.$$

The assertion for $t = 0$ follows by continuity. Thus Q is positive semidefinite.

For the rational decomposition, choose a positive diagonal pivot $d = Q_{kk}$. Completing the square gives

$$z^T Q z = d \left(z_k + \sum_{j \neq k} \frac{Q_{kj}}{d} z_j \right)^2 + z_R^T \left(Q_{RR} - \frac{Q_{Rk} Q_{kR}}{d} \right) z_R.$$

The residual Schur complement is positive semidefinite, and every entry remains rational. Repeat. If all remaining diagonal entries vanish in a positive semidefinite matrix, all remaining off-diagonal entries also vanish: otherwise a suitable vector supported on two coordinates gives a negative quadratic form. The process therefore terminates with the claimed rational weighted squares. $\square$

The algorithm rejects a negative diagonal pivot or a nonzero zero-diagonal residual. A returned certificate is then checked again by polynomial expansion. No numerical eigenvalue threshold is used. A matrix whose entries include a tiny nonzero rational off-diagonal value is treated exactly, not rounded to a positive semidefinite matrix.

7.2 Pseudocode and cost

```
quadratic_certificate(p):
  if total_degree(p) > 2: return Unknown
  Q := exact_homogenized_matrix(p)
  active := all coordinates
  terms := []
  while active is not empty:
    if any active diagonal is negative: return Unknown
    choose a positive diagonal pivot d
    if no pivot exists:
      if active submatrix is zero: break
      else: return Unknown
    emit d * (pivot_variable + rational_linear_tail)^2
    Q := exact Schur complement on remaining coordinates
    certificate := sum of emitted terms
  return certificate only if expand(certificate) == p
```

The matrix arithmetic uses $O(n^3)$ rational operations, but that is not an $O(n^3)$ bit-complexity claim. Numerators and denominators can grow substantially. Sparse pivots, fraction-free variants, and explicit bit-length budgets are natural production improvements.

A failed global quadratic test does not imply the constrained goal is false. For example, x is not globally nonnegative but is nonnegative under the hypothesis $x \geq 0$. Such a result should route to a constraint-aware worker, not terminate the entire tactic.

7.3 Why this adds a useful mechanism

Expanding a nonnegative quadratic can hide the linear forms whose squares establish its sign. Existing arithmetic preprocessing may discover the right consequences, but it need not start with those forms. This worker systematically reconstructs a global quadratic certificate rather than hoping a useful square already appears syntactically.

The implementation does not claim that all 60 generated quadratic examples defeat **grind** or **nlinarith**. Its component ablation only shows that recognizing positive expanded monomial squares is much weaker than reconstructing arbitrary rational affine squares on this constructed family.

8 Algorithm B: finite-cone search with exact repair

Status: Implemented and tested; not a general SDP solver.

8.1 Candidate construction

Select a bounded list of candidate columns

$$f_k = q_k^2 \prod_i g_i^{a_{ki}}.$$

The initial candidate grammar uses constants, variables, selected sums or differences, products occurring in the target, and low-degree combinations of known bounds. A target-driven version can add differences $x - y$ when a negative xy coefficient suggests a missing square, or factors associated with a guarded cancellation.

Expand the columns in a common monomial basis. If M is the resulting coefficient matrix and b is the target coefficient vector, search for

$$M\lambda = b, \quad \lambda \geq 0.$$

The prototype uses SciPy's `linprog(method='highs')` for this numerical proposal [10]. Its objective slightly prefers smaller candidate expressions, but feasibility rather than optimality is the mathematical issue.

8.2 Repair instead of rounding a proof

A floating result is not a certificate. The prototype selects the support of the proposed coefficient vector, solves the corresponding coefficient equations again over exact rationals, requires nonnegative repaired coefficients, and finally expands the certificate to check the original target identity.

This separation handles the most important failure mode: a numerical solver can consider a small nonzero residual acceptable, while an identity proof cannot. A rational coefficient that is merely close to a valid one is still wrong. Numerical infeasibility is also not a proof that the goal is false, or even a formally established proof that this finite cone contains no solution.

The current repair procedure is deliberately modest. It takes coefficients above a fixed floating support threshold and sets free parameters of the rational linear system to zero. This can miss valid certificates with small coefficients or a different feasible choice of free parameters. The correct result is **Unknown**, not a weakened acceptance test. A better oracle can explore nearby supports, retain small values, and solve exact feasibility in the nullspace without changing the checker.

8.3 A domain-dependent example

Under $0 \leq x \leq 1$ and $0 \leq y \leq 1$,

$$2x(1-x) + 3y(1-y) + xy + \frac{1}{5} \geq 0.$$

The target may be supplied in expanded form, with negative x^2 and y^2 coefficients. The useful certificate consists of products of the four bound polynomials $x, 1-x, y, 1-y$. Global SOS search alone cannot establish this polynomial's nonnegativity because the assertion is domain-dependent.

The experiment includes this example and verifies that removing or changing its hypotheses causes the exact checker to reject the certificate. Hypothesis provenance is therefore part of the tested interface, although full Lean local-context management is not implemented in Python.

8.4 A concrete path to general SOS search

A later worker can replace the fixed square list with Gram matrices. Choose a monomial vector v_D and seek matrices $Q_\alpha \succeq 0$ satisfying coefficient identities for

$$p = v_D^T Q_0 v_D + \sum_{\alpha} v_{D_\alpha}^T Q_\alpha v_{D_\alpha} g^\alpha + \sum_j u_j h_j.$$

CVXPY exposes semidefinite constraints and can serve as a modeling front end [12]. Its output remains untrusted. Recover rational matrices inside the exact affine coefficient-constraint space, then perform an exact positive-semidefiniteness check and convert each matrix to rational weighted squares.

Entrywise rounding followed by approximate eigenvalue testing is not acceptable. Singular Gram matrices are especially delicate: perturbation can destroy either positivity or the coefficient identity. Exact nullspace handling, support changes, and facial reduction are search techniques worth implementing, but acceptance must still reduce to (3). The package contains no SDP implementation and no SDP performance claims.

9 Algorithm C: exact Bernstein subdivision on boxes

Status: Implemented, tested, and used to emit a 32-leaf Lean replay candidate.

9.1 Coefficient formula

Let p be a polynomial on the rational box

$$X = \prod_{i=1}^n [l_i, u_i], \quad l_i < u_i.$$

Substitute $x_i = l_i + (u_i - l_i)t_i$ to obtain

$$q(\mathbf{t}) = \sum_{\alpha} a_{\alpha} \mathbf{t}^{\alpha}, \quad 0 \leq t_i \leq 1.$$

Let d_i be the coordinatewise degree of q . Its tensor-product Bernstein expansion is

$$q(\mathbf{t}) = \sum_{0 \leq \beta \leq \mathbf{d}} b_{\beta} \prod_i \binom{d_i}{\beta_i} t_i^{\beta_i} (1 - t_i)^{d_i - \beta_i},$$

where the exact rational coefficients are

$$b_{\beta} = \sum_{\alpha \leq \beta} a_{\alpha} \prod_i \frac{\binom{\beta_i}{\alpha_i}}{\binom{d_i}{\alpha_i}}. \quad (4)$$

Lemma 9.1. For $0 \leq a \leq d$,

$$t^a = \sum_{b=a}^d \frac{\binom{b}{a}}{\binom{d}{a}} \binom{d}{b} t^b (1 - t)^{d-b}.$$

Proof. Use $\binom{d}{b}\binom{b}{a} = \binom{d}{a}\binom{d-a}{b-a}$, factor out t^a , and apply the binomial theorem to the remaining sum. $\square$

Applying this identity in each coordinate proves (4). Each basis term is nonnegative on the unit box, and the basis terms sum to one. Consequently

$$\min_{\beta} b_{\beta} \leq p(\mathbf{x}) \leq \max_{\beta} b_{\beta} \quad (\mathbf{x} \in X).$$

In particular, nonnegative coefficients certify nonnegativity throughout the box.

9.2 The subdivision certificate

If a box does not certify immediately, split a longest side at its exact rational midpoint. A certificate is a binary tree. An internal node contains an axis, an interior rational split point, and both children. A leaf contains a nonnegative rational lower bound.

The checker independently reconstructs every child box, recomputes each leaf's coefficients, and confirms that the claimed lower bound does not exceed the smallest coefficient. Requiring both children proves coverage. Overlap on a shared face is harmless; missing an entire child is not. The test suite corrupts split points, removes children, and supplies excessive lower bounds to verify rejection.

The search uses maximum depth and node limits. It may evaluate the polynomial at the midpoint to abandon a hopeless box early, but it never returns that floating or sampled evaluation as a Lean refutation. All implemented evaluations are rational; the API still returns only a certificate or Unknown.

9.3 Limits and the exact-zero obstruction

The tensor size is $\prod_i (d_i + 1)$, and subdivision can be exponential. This method is best for moderate dimension, low coordinatewise degree, and meaningful compact bounds. The implementation limits tensor coefficients and tree nodes, but it is research code, not a complete defense against hostile memory-exhaustion inputs.

For a polynomial strictly positive on a compact box, sufficiently fine subdivision can in principle establish positivity. Informally, on a shrinking box every Bernstein coefficient approaches the polynomial value at the box location; polynomial expansions give uniform control, while compactness supplies a positive minimum. This does not supply a useful bound for every input and does not justify ignoring the configured caps.

Zeros behave differently. For a nonzero polynomial with an interior zero, an all-nonnegative Bernstein expansion on that box is impossible: every basis function is strictly positive at an interior point, so a zero value would force every coefficient to vanish. Thus a finite dyadic subdivision cannot certify $(x - 1/3)^2$ on $[-1, 1]$ by nonnegative leaf coefficients alone. Some leaf must contain $1/3$ in its interior. This is a mathematical obstruction, not merely a poor choice of search depth.

It motivates the exact-zero-aware univariate worker in the next section. In several variables, candidate factorization, algebraic equality reasoning, and constraint-relative certificates can play an analogous role. A generic multivariate zero-set treatment is not part of the prototype.

9.4 A nontrivial box example

Define

$$M(x, y) = x^4y^2 + x^2y^4 - 3x^2y^2 + 1.$$

The experiment certifies $M + 1/16$ on three domains. On $[-1, 1]^2$, one Bernstein leaf suffices; its minimum coefficient is exactly $1/16$. On $[-2, 2]^2$, the search constructs 32 leaves at maximum depth six. On $[-3, 3]^2$, it constructs 56 leaves at maximum depth nine.

The generated `BernsteinReplay.lean` expands the 32-leaf certificate into ordinary case splits, local bound proofs, nonnegative Bernstein terms, and ring identities. It does not require trusting the Python tree search. It is nevertheless only a candidate Lean proof until compiled successfully.

The first development test incorrectly expected the smallest domain to require subdivision. The exact coefficient calculation showed that the test assumption was wrong. The corruption test was moved to the larger domain, and independent SymPy reconstruction tests were added. No positivity condition was weakened to make the test pass.

10 Algorithm D: univariate certificates that preserve zeros

Status: Implemented and tested in Python; no generic Lean Sturm replayer included.

10.1 Square-factor reduction

For a nonzero rational univariate polynomial, extract even multiplicities:

$$p(x) = q(x)^2 r(x),$$

where r is square-free and its rational scalar coefficient retains the original sign. Full factorization is not necessary in principle: square-free decomposition is enough. The small prototype uses SymPy's rational factorization interface for convenience, then verifies the resulting identity with its independent rational polynomial representation [11].

A certificate on a rational interval $[a, b]$, $a < b$, consists of q , r , and a Sturm chain for r . The checker requires:

1. the exact identity $p = q^2 r$ and a valid, terminating signed-remainder chain for r ;
2. no roots of r in the *open* interval (a, b) ;
3. $r((a + b)/2) > 0$ and $p(a), p(b) \geq 0$.

The zero polynomial is handled by $q = 0, r = 1$.

10.2 The checked Sturm chain

The chain starts with $r_0 = r$, $r_1 = r'$, then follows

$$r_{i+1} = -\text{rem}(r_{i-1}, r_i)$$

until the next remainder is zero. Constant r has a one-element chain. The checker recomputes the derivatives and rational remainders, requires exact equality with the supplied chain, and requires its last element to be a nonzero constant. The last condition establishes that r is square-free.

Let $V(t)$ denote the number of sign changes in the chain. For endpoint roots, ordinary substitution followed by a casual zero-removal convention is easy to mishandle. The implementation computes *one-sided* signs. At $a+$, the sign of a chain polynomial is the sign of its first nonzero derivative

at a ; at $b-$, multiply that sign by $(-1)^k$ when the first nonzero derivative has order k . The open-interval root count is

$$N_{(a,b)}(r) = V(a+) - V(b-).$$

Independent tests compare root counts with SymPy and explicitly exercise rational roots at both endpoints.

10.3 Soundness and fragment completeness

Theorem 10.1 (Univariate interval certificate). *A certificate satisfying the conditions above proves $p(x) \geq 0$ for every $x \in [a, b]$.*

Proof. The Sturm count establishes that r has no root in (a, b) . By continuity, its sign is constant there; the positive midpoint value fixes that sign as positive. Hence $q^2r \geq 0$ on the interior. The exact identity and the two endpoint checks complete the proof on the closed interval. $\square$

This proof uses the mathematical Sturm theorem. Its use is justified in the algorithm specification and exercised by tests, but not formalized in Lean in this archive. A production Lean implementation must prove or integrate that theorem and check the instantiated certificate before treating this worker as trusted.

Ignoring degree and resource caps, exact square-free decomposition plus this check is complete for nonnegativity of rational univariate polynomials on a rational closed interval. Indeed, if nonzero $p = q^2r$ is nonnegative, a simple interior root of square-free r would change the sign of p , which is impossible. Thus r has no interior root and is positive there. Conversely, the certificate theorem applies. The implementation still returns `Unknown` for rejected candidates rather than constructing a formal counterexample.

10.4 Why this worker is useful alongside Bernstein search

The polynomial $(x - 1/3)^2$ is handled without aligning a dyadic split with its zero. So is $(x^2 - 2)^2$, whose real zeros are irrational. The constructed test family multiplies rational and irrational even-root factors by a positive quadratic and the endpoint factors $(x + 2)(2 - x)$ on $[-2, 2]$. All 30 instances are certified by the square-factor/Sturm worker; none is certified by the depth-12 Bernstein-only ablation.

This is an actual component-level distinction. It does not imply that these examples are difficult for an existing Lean tactic once a factorization is supplied. The contribution is an explicit search-and-check route for finding and validating the relevant algebraic structure.

11 Theory exchange, guarded normalization, and equality search

11.1 Certified product envelopes

Bound propagation can generate inexpensive facts before invoking a larger nonlinear search. If $l_x \leq x \leq u_x$, $l_y \leq y \leq u_y$, and $z = xy$, then four products of nonnegative bound differences give

$$\begin{aligned} z &\geq l_x y + l_y x - l_x l_y, \\ z &\geq u_x y + u_y x - u_x u_y, \\ z &\leq u_x y + l_y x - u_x l_y, \\ z &\leq l_x y + u_y x - l_x u_y. \end{aligned}$$

For instance, the first inequality is the expansion of $(x - l_x)(y - l_y) \geq 0$. These facts are linear in x, y, z and can be consumed immediately by the existing arithmetic engine. Their derivations, bounds, and the equation $z = xy$ must accompany them.

This worker is specified but not separately implemented in the prototype. It should be prioritized because its certificates are small and its output is useful across theories. It also offers a natural trigger: run only when the bounds of a product operand improve.

11.2 A complete cross-theory proof shape

Suppose

$$p = x^2 - 2xy + y^2, \quad p \leq 0,$$

and the target is $f(p) = f(0)$ for an otherwise uninterpreted $f : \mathbb{R} \rightarrow \mathbb{R}$. The nonlinear worker discovers $p = (x - y)^2$, proves $0 \leq p$, and exports that inequality. Antisymmetry gives $p = 0$; congruence gives the target. The supplied candidate Lean script spells out this proof without requiring a new theory of f .

A production event-driven implementation would allow the quadratic worker to provide the sign fact, the linear/order worker to provide the equality, and **grind** to close the congruence goal. No single worker needs to solve the entire problem. This is the central integration pattern, not an assertion that stock **grind** fails on this small example.

11.3 Bounded equality search

Equality saturation should be used selectively because **grind** already supplies congruence machinery. The proposed extension maintains small collections of alternative forms for terms relevant to a blocked obligation. It prioritizes forms that expose recursive calls, match a theorem conclusion, or reduce an arithmetic certificate's dimension. Rewrites are justified by equality proofs; conditional rewrites create explicit guards.

Do not saturate associativity, commutativity, and distributivity indiscriminately. Existing normalizers already handle much of that structure more efficiently. Use algebraic normal forms for certificate checking, while retaining a few factored or recursive views for search. Limit term count, expansion degree, and rewrite generations. If a conditional rule depends on a proposition not yet proved, its output belongs in the obligation graph, not in an unconditional equality class.

11.4 What a richer system still needs

General quantified semialgebraic formulas, induction over arbitrary well-founded relations, synthesis of sophisticated recursive witnesses, and analytic inequalities involving transcendental functions require further specialized procedures or library theorems. The proposed architecture can host them, but the presence of a worker interface does not implement them. The initial scope is deliberately chosen so that the certificate formats and proof obligations are explicit, finite, and testable.

12 Libraries, modules, and implementation boundaries

12.1 Use existing libraries for what they already do well

The proposed implementation should use Lean's metaprogramming framework for elaboration, local contexts, metavariable goals, induction, and proof construction. A discrimination-

tree index and an environment extension can store premise-search metadata. Internal `Lean.Meta.Tactic.Grind` access belongs behind a small adapter because source compatibility is not guaranteed between Lean releases [3, 2].

Mathlib supplies normalization and proof-producing leaf tactics; the important initial components are `ring`, `linarith/nlinarith`, `positivity`, `norm_num`, and cast normalization [8, 6, 7]. Use them to reconstruct small certificates rather than introducing a second independent arithmetic proof language prematurely. For larger certificates, a reflected checker becomes worthwhile once its soundness theorem is established.

Aesop can supply or inform structural proof-search infrastructure. Duper can be an optional, separately pinned saturation worker over a small retrieved premise set [4, 5]. Neither should become a mandatory dependency for the exact arithmetic checker. Dependency conflicts and toolchain mismatches should disable the optional worker with an explicit diagnostic rather than break the core tactic.

The executable Python package uses `fractions.Fraction` and a small sparse polynomial implementation for checking. SciPy/HiGHS is used only by the finite-cone oracle. SymPy is used by univariate search and independent tests, not by the standard-library-only certificate rechecker. CVXPY is a proposed SDP front end, not an installed or tested dependency of this prototype [10, 11, 12].

12.2 A suggested Lean module layout

The following paths are an implementation plan, not files claimed to exist in the archive:

Forge/Frontend.lean	-- syntax, configuration, diagnostics
Forge/Core/GoalGraph.lean	-- AND/OR obligations and proof assembly
Forge/Core/Facts.lean	-- scoped propositions, proof dependencies
Forge/Core/Scheduler.lean	-- deterministic quotas and budgets
Forge/Grind/Adapter.lean	-- pinned-version interface to grind
Forge/Arithmetic/Reify.lean	-- typed polynomial views and bridge proofs
Forge/Arithmetic/Cone.lean	-- cone semantics and exact replay
Forge/Arithmetic/Quadratic.lean	-- quadratic search adapter
Forge/Arithmetic/Bernstein.lean	-- coefficients, coverage, replay
Forge/Arithmetic/Sturm.lean	-- verified univariate root-count checker
Forge/Structural/Induct.lean	-- scheme and generalization search
Forge/Structural/Witness.lean	-- typed witness grammars and obligations
Forge/Oracle/Protocol.lean	-- bounded certificate data, never code
Forge/Replay/AxiomPolicy.lean	-- dependency/axiom audit

The Python modules in the archive mirror the arithmetic and synthesis responsibilities, but do not implement the Lean goal graph, local-context API, or metaprogramming adapters. Keeping the boundaries explicit is preferable to shipping a thin tactic macro and describing it as the complete system.

12.3 A proposed user interface

A useful terminal tactic should have a small default interface, an explanatory mode, and explicit controls for expensive or trust-changing workers. For example, the following is *proposed syntax only*:

```

forge
forge? [selectedLemma, recursiveDefinition]
forge (config := {
  mode := .compatibility
  inductionDepth := 2
  witnessDegree := 1
  polynomialDegree := 8
  externalSearch := true
  trustMode := .kernelOnly
})

```

A successful explanatory invocation should emit a smaller replay plan: the chosen induction scheme, generalized variables, witness expressions, necessary library lemmas, and concrete certificate data. It should not require the original broad search again. On failure it should identify the first substantive blocked obligation or exhausted budget, not print thousands of irrelevant facts.

Parameters must describe real implementation limits. For example, a polynomial-degree bound and a certificate monomial bound control different sources of explosion and should not be conflated. A trust option must not silently change because a faster worker happens to be available.

12.4 The external certificate protocol

Exchange structured data, not executable code. The problem request should contain a versioned schema, registered variable identifiers, exact rational coefficients, normalized relation kinds, input bounds, and a resource envelope. Returned certificates may reference only registered expressions and hypotheses. The Lean side reconstructs the original semantics from its own registry.

The parser must reject malformed dimensions, duplicate or noncanonical monomials when canonical form is required, negative exponents, oversized integers, unknown references, invalid tree splits, and unsupported certificate variants. It should not use `eval`, run a returned script, load arbitrary shared libraries, or trust a theorem name supplied by the external solver.

The packaged JSON decoder demonstrates modest checks on dimensions, sizes, rational encodings, and certificate structure. It is not a hardened sandbox. In production, enforce limits *before* expensive polynomial expansion as well as after it, and isolate search processes from the proof-checking process. A denial of service is not a soundness failure, but it still makes a tactic unusable.

13 Trust: what exactly is being checked?

13.1 Three distinct layers

It is essential to distinguish mathematical certificate soundness, software correctness of a Python checker, and a proof accepted by Lean. The propositions in this report explain why the certificates imply the desired statements. Tests provide evidence about the Python implementation. Neither alone supplies a Lean theorem. A completed tactic must reconstruct a Lean proof or use a formally verified checker whose result is established inside Lean.

For the prototype, `python -S verify_certificates.py` rechecks all 198 stored certificates without loading site packages. This demonstrates independence from the numerical and symbolic search libraries. It does not eliminate trust in Python, the handwritten checker, or the host execution environment, and is not described as kernel verification.

13.2 Axiom auditing is part of acceptance

The Lean kernel verifies derivations relative to the axioms they use. A strict automation policy should therefore inspect the transitive axiom dependencies of the final proof and its auxiliary declarations. A practical whitelist may allow the project’s usual logical axioms, such as propositional extensionality, quotient soundness, and classical choice when permitted, while rejecting admitted proofs and compiler-result axioms.

The current tactic reference explicitly documents that `native_decide` and `bv_decide` rely on native code generation and associated trusted results. Thus “the SAT solver returns a checked certificate” does not by itself mean “no compiler trust is added”. FORGE should put that route behind an opt-in trust mode rather than enabling it under a kernel-only label [15].

A kernel-only bitvector route would require replaying a supported certificate checker through a reduction/proof mechanism that does not introduce native-result axioms, and measuring the resulting cost. That route is proposed, not implemented here. A project willing to accept native-code trust can use the existing worker, provided the trust profile is visible in the trace and output metadata.

13.3 Diagnostic holes are not accepted proofs

The inspected `grind` configuration includes an option concerning admitted branches in trace-oriented diagnostics [2]. That is not a claim that ordinary successful `grind` proofs are unsound. It is a reminder that a coordinator must never mistake a diagnostic artifact for a finished proof. The root declaration’s dependencies must be checked for admitted terms even when an inner tool reports a successful diagnostic operation.

The candidate Lean files in this archive deliberately contain no admitted theorem bodies or new oracle axioms. Because they were not compiled here, their status remains *candidate source*, not “kernel checked”. The distinction is recorded in the README and status metadata.

13.4 The proof-cost budget

Search can produce a valid but impractically large certificate. Track certificate term count, proof-DAG size, replay time, and axiom-audit time. Share repeated polynomial normalizations and repeated nonnegativity proofs. Prefer sparse certificates and small rational coefficients when several valid certificates are available, but never alter a certificate approximately merely to reduce its size.

When a proof is rejected for size or replay budget, return a useful `Unknown` diagnostic and preserve the candidate certificate as an optional debugging artifact. Do not accept a partial proof or silently raise the trust level to finish the goal.

14 Executed experiments

14.1 Environment and reproducibility

The Python experiments ran under Python 3.13.5 on the Linux environment described in `results/experiments.json`, using NumPy 2.3.5, SciPy 1.17.0, SymPy 1.14.0, and pytest 9.0.2. These are the versions actually executed, not the versions advertised by the latest online documentation. The certificate checkers use exact rational arithmetic.

No `lean`, `lake`, or `elan` executable was available in the environment. Direct network access

from the execution container was also unavailable, so no local Lean installation or compilation is claimed. Public documentation and GitHub sources were inspected through the available research tools. The candidate replay project pins Lean v4.34.0 and Mathlib commit

1cf325a0cf67aca2b04d76b5380ff6a9e410aefa

whose `lean-toolchain` file specifies that Lean version [16]. Optional Aesop or Duper integration is not part of this replay project.

14.2 Unit, mutation, and independent tests

The final pytest run reports **275 passed**. These are test instances, not 275 Lean theorems. Some tests contain several assertions. The suite covers exact arithmetic identities, rational quadratic decompositions, finite-cone repair, conditional certificates, Bernstein coverage, recurrence formulas, affine witnesses, univariate multiplicities, invalid inputs, and explicit `Unknown` cases.

Mutation checks change targets, weights, hypotheses, recurrence base terms, affine offsets, Bernstein split points, child nodes, claimed bounds, and Sturm chains. The checker must reject such changes when they invalidate the certificate. Independent SymPy tests reconstruct Bernstein expansions symbolically, expand quadratic certificates, and compare univariate root counts. These cross-checks reduce dependence on one implementation’s internal representation, but do not amount to a formal verification of the code.

The tests also preserve limitations as regression cases. A degree-limited recurrence search can fail on a valid higher-degree answer. Integer affine synthesis rejects a merely rational witness. Bernstein-only search fails at a non-dyadic interior zero. A higher-degree polynomial is outside the exact quadratic worker. Returning `Unknown` in these situations is expected behavior.

14.3 Constructed component benchmark

The benchmark contains 198 deterministic instances. Many are generated from known certificate families by construction, and the same mechanism families are used in tests. There is no held-out theorem corpus. Their purpose is to establish reproducibility, exercise the search-to-check interfaces, and expose the effect of specific mechanisms, not estimate performance on Mathlib.

Each case is searched three times after numerical-library warm-up. Table 2 reports the median of the per-case median search times and the corresponding explicit recheck times. A search may already perform internal validation, so the recheck column is not the amount of checking hidden inside the search column. These are local Python timings, not Lean compilation or kernel-checking times.

Component	Certified	Ablation	Search ms	Recheck ms
Quadratic SOS	60/60	1	0.864	0.701
Finite cone LP	17/17	0	1.699	0.295
Bernstein box	18/18	1	0.685	0.389
Polynomial recurrence	33/33	—	2.513	0.403
Integral affine witness	40/40	—	0.074	0.027
Univariate with zeros	30/30	0	1.930	0.505

Table 2: Executed Python mechanism experiments. Ablations are restricted versions of these prototype mechanisms, **not** `grind`. A dash means no ablation was run.

The quadratic ablation only recognizes an expanded nonnegative combination of even monomials. The finite-cone ablation removes richer square candidates or, for the domain example, the hypothesis products. The Bernstein ablation disallows subdivision. The univariate ablation runs dyadic Bernstein search to depth 12 without factorization. These are deliberately narrow mechanism tests, not competitive theorem-prover baselines.

All returned benchmark certificates are stored in `results/certificates.json`. A separate run of the standard-library-only decoder and checker accepted **198/198**. Raw per-instance measurements, certificate sizes, leaf counts where applicable, and family summaries are included. Re-running the script will change timings but should reproduce the algebraic outcomes.

14.4 What the results establish

They establish that the supplied implementations actually synthesize certificates in six distinct fragments; that certificate replay is separable from the search libraries; that the specified mutation tests are rejected; and that the exact-zero-aware worker solves examples where the chosen subdivision-only mechanism is structurally insufficient.

They do *not* establish that FORGE is implemented as a Lean tactic, that any candidate Lean script compiles, that the certificates are checked by Lean, that the methods outperform existing Lean tactics, or that their success rate transfers to user developments. Those claims require different experiments. The system design is intended to produce substantially more capable automation, but the measured evidence here is component-level.

15 How to evaluate an actual Lean implementation fairly

15.1 Baselines and information access

At minimum compare stock `grind`, tuned `grind` with explicit budgets and relevant suggestion settings, a simple public-tactic portfolio, and the full FORGE system. On suitable subsets include Aesop, `nlinarith`, `omega`, and Duper where they are applicable. The comparison should not handicap `grind` by withholding library annotations that the new tactic receives.

Use the same imports, local context, theorem statement, typeclass environment, and available premise universe. If the new system performs premise retrieval, provide a matched retrieval condition to the baseline or report the retrieval effect separately. Do not compare stock search time to replay of a certificate precomputed by the new method: either count that search cost or label the experiment as replay-only.

15.2 A useful corpus design

Build a mixture of existing development goals and newly written examples. Stratify by recursive proofs, arithmetic inequalities, existential constructions, equality-plus-arithmetic interactions, and already-easy baseline goals. Include valid but unsupported problems and false conjectures to exercise safe failure.

Split by theorem dependency families or development modules, not random near-duplicate statements. Freeze retrieval indexes and policy tuning before the held-out run. Exclude the target theorem, its equivalent aliases, and direct answers introduced by the benchmark harness from the premise universe. Imported mathematical theorems that genuinely belong to the environment are allowed, but both systems must have the same opportunity to use them.

The synthetic examples in this archive are useful unit tests and regression tests. They should not become the only evidence for a broad claim of mathematical automation progress.

15.3 Resources and metrics

Report total CPU time, wall time, peak memory, heartbeats where relevant, external-process time, replay time, proof size, and axiom dependencies. Distinguish cold import costs from warm elaboration costs. Run methods in isolated processes so a previous theorem’s auxiliary declarations and caches do not leak into another method’s state.

The primary outcome is the number of accepted proofs under an identical total resource budget. Report paired wins and losses, not just total solved counts. Use paired confidence intervals or a paired test for success differences, and inspect regressions by problem family. For times, use budget curves or censored-runtime summaries rather than averaging only the successful cases and ignoring timeouts.

Compatibility mode has a different question: how much additional resource rescues failures without replacing the successful baseline run? Plot rescue coverage against that *additional* budget. Never present its success-set-preservation property as an equal-cost performance theorem.

15.4 Ablations that would test the actual design

Remove one capability at a time: backward obligation production, induction generalization, affine witnesses, hidden-quadratic certificates, hypothesis-product cones, Bernstein subdivision, zero-aware univariate search, cross-worker fact exchange, and relevance-based scheduling. In particular, compare a noncommunicating portfolio with the same workers against the shared-fact coordinator. That experiment tests whether integration, rather than just adding more solvers, delivers the intended benefit.

A possible engineering acceptance gate is a material positive rescue rate on a preregistered held-out set while maintaining a controlled regression rate on the full corpus. A numerical threshold should be selected for the intended workload before running the test. This report does not predict or claim a particular percentage improvement.

16 Implementation sequence and acceptance gates

16.1 Milestone 1: trustworthy orchestration

Implement the frontend, deterministic compatibility mode, full state rollback, worker contracts, and axiom-policy audit. Add an end-to-end test where a deliberately malformed proof or admitted theorem is rejected. Add a regression test showing that a failed branch cannot leak a local equality into its sibling. At this stage, only existing proof-producing leaf tactics need run.

16.2 Milestone 2: exact algebraic replay

Implement typed reification and cone replay in Lean. Port or call the Python quadratic and finite-cone oracles through a bounded data protocol. Compile the generated examples and replace any fragile tactic scripting with direct theorem applications where practical. Acceptance requires successful Lean checking of every claimed end-to-end example, mutation rejection at the data boundary, and recorded axiom dependencies.

This stage can already add useful nonlinear facts to a stock `grind` leaf. It need not wait for a complete custom induction planner or general SDP search.

16.3 Milestone 3: structural planning and witnesses

Add explicit theorem-application obligations, affine witnesses, and bounded induction with dependency-directed generalization. Implement the accumulator example, natural-number polynomial recurrences, and a small family of recursive data-structure proofs. Check that synthesized lemmas are proved before they are shared and that failed structural alternatives restore the original goal.

An induction “success” should include the selected principle and a reconstructed motive in its replay trace. A witness “success” should include the witness term and every bound, integrality, and typeclass obligation needed for it.

16.4 Milestone 4: domain-aware positivity

Formalize the Bernstein coefficient and coverage rules, then add compact-box certificates. Integrate a verified Sturm/root-count result or prove the required checker soundness theorem before accepting the univariate certificates in Lean. The univariate part is a substantial verification task and should not be hidden behind an unchecked foreign call.

The initial explicit 32-leaf replay can validate the Bernstein search-to-source path, but production acceptance should prefer a reusable checked certificate theorem over a large amount of duplicated tactic text when that improves proof size and reliability.

16.5 Milestone 5: efficient cooperation

Add incremental fact subscriptions, proof-DAG sharing, and the version-pinned `grind` adapter. Implement bound-derived product envelopes and target-driven certificate candidates. Measure whether repeated leaf restarts are a significant cost before investing in invasive internal hooks. Keep a terminal-tactic fallback so a Lean upgrade need not disable the entire system.

16.6 Milestone 6: optional broader search

Only after the preceding layers are reliable, add Duper premise search, SDP proposals with exact reconstruction, or a learned ranking policy. Learned components may choose which candidate to try; they do not decide which statements are accepted. Native-trust bitvector workers remain opt-in. Every optional dependency has its own toolchain and feature-compatibility check.

17 Reproducing and extending this package

17.1 Python commands

From the archive root, create an isolated Python environment and install the pinned dependencies in `prototype/requirements.txt`. Then run:

```
cd prototype
python -m pytest -q
python run_experiments.py
python -S verify_certificates.py
python emit_lean.py
```

The first command exercises the final test suite. The second overwrites the experiment JSON/CSV files with new timings. The third rechecks stored certificates without numerical or symbolic third-party libraries. The last regenerates the uncompiled Lean candidates. The exact checkers can also be imported directly without installing SciPy or SymPy; only the corresponding search functions import those libraries.

17.2 Lean candidate compilation

In an environment with the pinned Lean toolchain available, the candidate project can be prepared and checked with:

```
cd lean
lake update
lake build
```

These commands were *not* run in the authoring environment. The project imports Mathlib and may need its dependencies and compiled artifacts before building. The supplied `run_checks.py` records a local build result without overwriting the original execution-status record. A failed compilation is evidence to fix the candidate source, not permission to replace a proof with an admission.

The generated candidates cover a hidden quadratic certificate, seven power-sum recurrence proofs, an integral affine witness, a 32-leaf Bernstein proof, an accumulator induction, a cross-theory equality, and a factored univariate sign proof. The latter is not a generic Lean replay of the Sturm checker.

17.3 Archive map

article/forge.tex	-- main article source
article/forge.pdf	-- rendered article
article/sections/	-- editable section sources
prototype/forge/	-- exact representations and algorithms
prototype/tests/	-- unit, mutation, independent tests
prototype/run_experiments.py	-- deterministic benchmark generator
prototype/verify_certificates.py	-- standard-library-only recheck
prototype/emit_lean.py	-- candidate Lean source generator
lean/	-- pinned, uncompiled replay project
results/	-- actual logs, data, certificates, status
README.md	-- instructions and limitations

18 Conclusion

The most promising route beyond **grind** is not to replace its existing equality and arithmetic infrastructure. It is to surround that infrastructure with proof-structure search and feed it new, independently validated mathematical consequences. The concrete components proposed here cover three complementary kinds of progress: discovering a certificate, discovering an invariant, and constructing a witness.

The arithmetic design is deliberately broader than one SOS tactic. Exact quadratic decomposition handles hidden global quadratic positivity. Finite cones exploit useful factors and hypotheses. Bernstein trees exploit compact domains. Square-factor/Sturm certificates handle univariate zeros that obstruct subdivision. Polynomial recurrence and affine-witness synthesis show how exact algebra can generate the structure of a proof, not only close its final arithmetic goal.

The supplied code makes these mechanisms testable: 275 test instances passed, 198 constructed problems yielded certificates, and every stored certificate was rechecked without the search libraries. The complete Lean tactic, generic Lean certificate checkers, and performance advantage over **grind** remain implementation and evaluation work, explicitly separated from the completed experiments. The result is a concrete, falsifiable design for substantially stronger automation, with working algorithmic building blocks and a precise account of what still has to be proved and measured.

References

- [1] Lean contributors. *The Lean Language Reference: The **grind** tactic*. Inspected 14 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Lean contributors. *Init/Grind/Config.lean*, Lean v4.34.0. Pinned implementation source. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Init/Grind/Config.lean>.
- [3] Lean contributors. *Lean/Meta/Tactic/Grind/Types.lean*, Lean v4.34.0. Pinned implementation source. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Types.lean>.
- [4] Aesop contributors. *Aesop: Automated Extensible Search for Obvious Proofs*. Repository documentation, including search organization and induction examples. Inspected 14 September 2026. <https://github.com/leanprover-community/aesop>.
- [5] Duper contributors. *Duper*. Repository README, proof-producing saturation, premise interface, and portfolio configuration. Inspected 14 September 2026. <https://github.com/leanprover-community/duper>.
- [6] Mathlib contributors. *Mathlib.Tactic.Linarith.Frontend*. Generated documentation, certificate search and reconstruction, nonlinear preprocessing. Inspected 14 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [7] Mathlib contributors. *Mathlib.Tactic.Positivity.Basic*. Generated documentation. Inspected 14 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Positivity/Basic.html.
- [8] Mathlib contributors. *Mathlib.Tactic.Ring.Basic*. Generated documentation. Inspected 14 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Ring/Basic.html.
- [9] John Harrison. Verifying nonlinear real formulas via sums of squares. In *Theorem Proving in Higher Order Logics (TPHOLs 2007)*, LNCS 4732, pp. 102–118, Springer, 2007. DOI: [10.1007/978-3-540-74591-4_9](https://doi.org/10.1007/978-3-540-74591-4_9). Author abstract and bibliographic record: <https://www.cl.cam.ac.uk/~jrh13/papers/sos.html>.
- [10] SciPy contributors. *scipy.optimize.linprog, method=highs*. Official API documentation. The executed package version was SciPy 1.17.0; the online reference may be newer. <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html>.
- [11] SymPy contributors. *Polynomials Manipulation Module Reference*. Official documentation for rational polynomial factorization, Sturm sequences, and root counting. Executed version: 1.14.0. <https://docs.sympy.org/latest/modules/polys/reference.html>.
- [12] CVXPY contributors. *Advanced Constraints*, including semidefinite matrix constraints. Official documentation. Proposed optional dependency; not used in the experiments. <https://www.cvxpy.org/tutorial/constraints/index.html>.
- [13] Marcus Rossel and contributors. *lean-egg*. Repository documentation and discontinuation notice. Inspected 14 September 2026. <https://github.com/marcusrossel/lean-egg>.
- [14] Evan Wang, Simon Chess, Sophie Szeto, and Theodore Meek. *Learned Interventions in Lean 4 grind*. arXiv:2607.22972v2, 28 July 2026. The related-work discussion uses the authors’ abstract, not an independent reproduction of their experiments. <https://arxiv.org/abs/2607.22972v2>.
- [15] Lean contributors. *The Lean Language Reference: Tactic Reference*. Entries for `native_decide`, `bv_decide`, and evaluation/trust behavior. Inspected 14 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [16] Mathlib contributors. *lean-toolchain*, revision `1cf325a0cf67aca2b04d76b5380ff6a9e410aefa`. This file specifies `leanprover/lean4:v4.34.0`. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/lean-toolchain>.